

ALGORITMI E STRUTTURE DATI

ANALISI DEGLI ALGORITMI ED ALGORITMI DI ORDINAMENO

AULA VIRTUALE DEL 04-05-2026

V 1.1

ANALISI AMMORTIZZATA

In molte situazioni pratiche, ci troviamo ad eseguire un algoritmo ripetutamente nel tempo. Si pensi, ad esempio, ai motori di ricerca su Internet vedono arrivare miliardi di interrogazioni al giorno, ognuna delle quali corrisponde all'esecuzione di un certo algoritmo di ricerca. In contesti di questo tipo che abbiamo a che fare con collezioni di dati soggetti a sequenze di interrogazioni ed aggiornamenti.

ANALISI AMMORTIZZATA

Useremo il termine **operazione di aggiornamento** per indicare una operazione che modifica il contenuto informativo di una collezione di dati. Viceversa parleremo di **operazione di interrogazione** per denotare una operazione che esplora i dati di una collezione senza modificarne il contenuto informativo.

L'**analisi ammortizzata** studia le prestazioni **medie** di una sequenza di operazioni su una collezione di dati, piuttosto che della singola esecuzione di un algoritmo come abbiamo visto finora.

ANALISI AMMORTIZZATA

Studiare il comportamento di un algoritmo su una sequenza di esecuzioni può darne una caratterizzazione più raffinata, in sostanza nell'analisi ammortizzata le probabilità non entrano in gioco in nessun modo.

Ad esempio, una singola esecuzione di un algoritmo potrebbe richiedere tempo $\Theta(n)$ nel caso peggiore, mentre n esecuzioni potrebbero richiedere tempo totale ancora $\Theta(n)$ nel caso peggiore, e non $\Theta(n^2)$ come potremmo pensare.

ANALISI AMMORTIZZATA

Una singola esecuzione potrebbe richiedere molto tempo per via di una configurazione particolarmente svantaggiosa dei dati su cui opera. Tuttavia, i dati stessi potrebbero configurarsi in modo molto più vantaggioso durante le esecuzioni successive, per cui in media il tempo richiesto per ciascuna esecuzione potrebbe essere di fatto molto più basso di quello che si avrebbe nel singolo caso peggiore. Per caratterizzare le prestazioni di un algoritmo in media su una sequenza di operazioni, parleremo del concetto di **costo ammortizzato**.

ANALISI AMMORTIZZATA

Def. Costo Ammortizzato: Definiamo il costo ammortizzato di un algoritmo su una sequenza di k operazioni come:

$$T_a(n) = \frac{T(n,k)}{k}$$

dove T(n,k) è il tempo totale richiesto dall'algoritmo nel caso peggiore per tutte le k operazioni su istanza di dimensione n.

ANALISI AMMORTIZZATA

Vi sono diversi metodi per calcolare il costo ammortizzato di un’operazione: fra quelli più noti, il metodo dei crediti e il metodo del potenziale. La scelta di un metodo piuttosto che un altro è una questione di convenienza rispetto al particolare algoritmo che si sta considerando.

ANALISI AMMORTIZZATA: IL METODO DEI CREDITI

In alcuni casi può accadere che il costo di un'operazione dipenda strettamente dall'esecuzione nel passato di altre operazioni: per esempio una delete può rimuovere elementi precedentemente inseriti da altrante insert. Il metodo dei crediti permette di calcolare il costo ammortizzato di una operazione senza dover entrare nel dettaglio di queste dipendenze, che possono essere, a volte, difficili da determinare con precisione. L'idea è quella di associare delle monete immaginarie (o crediti) agli oggetti di una collezione. Queste monete, depositate da alcune operazioni, saranno in grado di «pagare» il costo di altre operazioni.

ANALISI AMMORTIZZATA: IL METODO DEI CREDITI

Assumeremo che:

1 moneta vale O(1) passi di calcolo

«come diceva Paperon de Paperoni «il tempo è denaro».

Chiaramente l’uso delle monete serve solo per l’analisi e non per il funzionamento.

ANALISI AMMORTIZZATA: IL METODO DEI CREDITI

Def. Metodo dei crediti: Il costo ammortizzato di una operazione secondo il metodo dei crediti è definito come:

$$T_a(n) = T(n) + deposito(n) - prelievo(n),$$

Dove T(n) è il numero di passi effettivo che l'operazione compie nel caso peggiore, deposito(n) è il numero di monete che deposita sugli oggetti, e prelievo(n) è il numero di quelle che preleva dagli oggetti.

ANALISI AMMORTIZZATA: IL METODO DEI CREDITI

In generale, assoceremo agli oggetti zero o più monete in base a qualche loro proprietà, e poi definiremo prelievo e deposito in modo che questa associazione venga rispettata tenendo conto di come gli oggetti stessi sono manipolati dalle operazioni.

ANALISI AMMORTIZZATA: IL METODO DEL POTENZIALE

Il metodo del potenziale è concettualmente simile al metodo dei crediti. La differenza principale è che, invece di associare crediti ai singoli oggetti, associamo piuttosto un potenziale ai dati nel loro insieme (in sostanza un numero non negativo). L'associazione avviene tramite una funzione potenziale $\Phi(I)$ che applicata ad una istanza I della struttura dati, definisce il potenziale dell'istanza.

ANALISI AMMORTIZZATA: IL METODO DEL POTENZIALE

Analogamente ai crediti, il potenziale di una collezione di oggetti può servire a pagare il costo di certe operazioni sulla collezione stessa. Il concetto è del tutto analogo a quello di un corpo in un campo elettrico o gravitazionale a cui è associata una **energia potenziale** che può essere trasformata in lavoro.

ANALISI AMMORTIZZATA: IL METODO DEL POTENZIALE

Def. Metodo del potenziale: Il costo ammortizzato di una operazione secondo il metodo del potenziale è definito come:

$$T_a(n) = T(n) + \Phi(I') - \Phi(I),$$

Dove T(n) è il numero di passi effettivo che l'operazione compie nel caso peggiore, I è l'istanza di dimensione n della struttura dati in ingresso all'operazione ed I' è l'istanza della struttura dati dopo l'esecuzione dell'operazione.

ANALISI AMMORTIZZATA: IL METODO DEL POTENZIALE

Mostriamo ora che sotto opportune condizioni, la somma dei costi ammortizzati secondo il metodo del potenziale fornisce una delimitazione superiore al tempo totale richiesto nel caso peggiore dell'intera sequenza:

Teorema. Siamo $I_0, \ldots, I_k$ le istanze di dimensione n della struttura dati ottenute su una sequenza di k operazioni a partire da un'istanza iniziale I_0. Se $\Phi(I_0) = 0$ e $\Phi(I_k) >= 0$, allora

T(n,k) <= $\sum_{i=1}^{k} T_a(n)$, dove T(n,k) è il tempo totale richiesto dall'algoritmo nel caso peggiore per tutte le k operazioni su istanze di dimensione n.

MODELLI AVANZATI: LA GERARCHIA DI MEMORIA

Per completezza, accenniamo ad un modello più sofisticato rispetto ai modelli presentati finora ma che meglio si presta ad astrarre una importante caratteristica delle moderne piattaforme di calcolo. Si parla infatti di gerarchia di livelli di memoria, ciascuno con un proprio costo e le proprie caratteristiche prestazionali.

E' quindi importate che i dati importati tramite un singolo accesso a disco, fintantoché si trovano in memoria centrale, siano «sfruttati» il più possibile dall'algoritmo.

MODELLI AVANZATI: LA GERARCHIA DI MEMORIA

In altre parole, se un algoritmo esibisce proprietà di località nell'accesso ai dati, il numero di I/O viene minimizzato, ottenendo prestazioni migliori. Per analizzare la caratteristica di località di accesso ai dati di un algoritmo, la classica macchina a registri è stata estesa in modo da supportare una visione a due livelli della memoria ed essere resa più realistica. Il modello di memoria esterna cattura quanto detto nel modo seguente:

MODELLI AVANZATI: LA GERARCHIA DI MEMORIA

- La memoria esterna ha una dimensione illimitata;
- La memoria interna ha una dimensione finita M;
- I dati sono trasferiti dalla memoria esterna a quella interna (e viceversa) in blocchi di dimensione B. Ossia si assume che $B < M/2$.

ANALISI DELLO SPAZIO (USO DELLA MEMORIA)

L’analisi dello spazio misura quanta memoria usa un algoritmo in funzione della dimensione dell’input n.

Due componenti principali

- Spazio input: memoria necessaria per memorizzare i dati (spesso non si conta)
- Spazio ausiliario: memoria extra usata dall’algoritmo

ANALISI DELLO SPAZIO (USO DELLA MEMORIA)

Algoritmo in-place

- Usa solo poche variabili
- Spazio: O(1)

```
def reverse(A):
    i, j = 0, len(A)-1
    while i < j:
        A[i], A[j] = A[j], A[i]
        i += 1
        j -= 1
```

ANALISI DELLO SPAZIO (USO DELLA MEMORIA)

Algoritmo con spazio extra

- Usa un altro array
- Spazio: O(n)

```
def copia(A):
    B = [0]*len(A)
    for i in range(len(A)):
        B[i] = A[i]
```

ANALISI DELLO SPAZIO (USO DELLA MEMORIA)

Attenzione alla ricorsione

Ogni chiamata ricorsiva usa stack:

- Fibonacci ricorsivo → O(n) spazio
- Merge sort → O(n) spazio

Punti di attenzione:

- Ottimizzare spazio può rendere l'algoritmo più efficiente in pratica
- A volte si scambia spazio per tempo (time–space tradeoff)

STABILITA' DEGLI ALGORITMI

Un algoritmo (di solito di ordinamento) è stabile se: mantiene l'ordine relativo degli elementi uguali. Esempio:
Input (valore, etichetta):
(5,A), (3,B), (5,C)
Dopo ordinamento per valore:

- Stabile (3,B), (5,A), (5,C)
- Non stabile (3,B), (5,C), (5,A)

Perché è importante? Quando gli elementi hanno più chiavi. Es: ordino prima per nome, poi per età. La stabilità è fondamentale quando i dati hanno informazioni implicite nell'ordine

CONFRONTO TRA EFFICIENZA DEGLI ALGORITMI

Serve per capire quale algoritmo è migliore per un certo problema.
Criteri principali

- Tempo (time complexity)O(1), O(log n), O(n), O(n log n), $O(n^2)$
- Spazio (space complexity)O(1), O(n), …

Algoritmo	Tempo medio	Spazio	Stabile
Bubble Sort	$O(n^2)$	O(1)	✔
Merge Sort	O(n log n)	O(n)	✔
Quick Sort	O(n log n)	O(log n)	✘

CONFRONTO TRA EFFICIENZA DEGLI ALGORITMI

Trade-off tipici

- Merge Sort:
  - veloce
  - usa memoria
- Quick Sort:
  - veloce e poco spazio
  - non stabile

CONFRONTO TRA EFFICIENZA DEGLI ALGORITMI

Non esiste "l'algoritmo migliore in assoluto"

Dipende da:

- dimensione input
- memoria disponibile
- necessità di stabilità
- caso medio vs peggiore

ALGORITMI DI ORDINAMENTO: BUBBLE SORT

L'idea di base è far "venire a galla" (come bolle) gli elementi più grandi verso la fine dell'array.

- **Come funziona:** L'algoritmo scorre l'array più volte. Ad ogni passaggio, confronta coppie di elementi adiacenti. Se sono nell'ordine sbagliato (il primo è maggiore del secondo), li scambia. Dopo la prima passata, l'elemento più grande si troverà sicuramente nell'ultima posizione. Il processo si ripete, ignorando via via gli ultimi elementi che sono già al posto giusto.
- **Complessità Temporale:** O(n^2) nel caso pessimo e medio. Se implementato con una flag che rileva se sono avvenuti scambi in una passata (ottimizzazione), il caso ottimo (array già ordinato) scende a O(n).

ALGORITMI DI ORDINAMENTO: BUBBLE SORT

- **Caratteristiche:** Fa moltissimi scambi ed è un algoritmo **stabile** (non altera l'ordine relativo di elementi con lo stesso valore).
- **Uso pratico:** Quasi nullo, serve principalmente a scopo didattico.

ALGORITMI DI ORDINAMENTO: BUBBLE SORT IN PYTHON 1/2

```python
# ============================================================
# BUBBLE SORT — Ordinamento a bolle
# Idea: confronta coppie adiacenti e "fa risalire" il massimo
# verso la fine dell'array, come bolle che salgono in superficie.
# ============================================================

def bubble_sort(arr: list) -> list:
    """
    Ordina una lista in-place con l'algoritmo Bubble Sort.

    Complessità temporale:
      - Caso migliore:  O(n)    — lista già ordinata (con ottimizzazione)
      - Caso medio:     O(n²)
      - Caso peggiore:  O(n²)   — lista in ordine inverso

    Complessità spaziale: O(1)  — ordinamento in-place
    Stabile: Sì (elementi uguali mantengono l'ordine relativo)
    """
    n = len(arr)

    for i in range(n):
        # Ottimizzazione: tiene traccia se avviene almeno uno scambio.
        # Se in un passaggio nessuno scambio avviene, la lista
        # è già ordinata e possiamo uscire anticipatamente.
        scambio_avvenuto = False
```

ALGORITMI DI ORDINAMENTO: BUBBLE SORT IN PYTHON 2/2

```
            # Dopo ogni passaggio esterno, gli ultimi i elementi
            # sono già nella posizione corretta → non servono confronti.
            for j in range(0, n - i - 1):
                # Confronta elemento corrente con il successivo
                if arr[j] > arr[j + 1]:
                    # Scambia i due elementi (unpacking Python)
                    arr[j], arr[j + 1] = arr[j + 1], arr[j]
                    scambio_avvenuto = True

            # Nessuno scambio → lista già ordinata, esci prima
            if not scambio_avvenuto:
                break

    return arr


# ─── Esempio d'uso ──────────────────────────────────────────
if __name__ == "__main__":
    dati = [64, 34, 25, 12, 22, 11, 90]
    print(f"Prima:  {dati}")
    bubble_sort(dati)
    print(f"Dopo:   {dati}")
    # Output:
    # Prima:  [64, 34, 25, 12, 22, 11, 90]
    # Dopo:   [11, 12, 22, 25, 34, 64, 90]
```

ALGORITMI DI ORDINAMENTO: SELECTION SORT

L'idea qui è dividere logicamente l'array in due parti: una parte ordinata (all'inizio) e una parte da ordinare (alla fine).

- **Come funziona:** Cerca l'elemento più piccolo in tutto l'array non ordinato. Una volta trovato, lo scambia con il primo elemento della parte non ordinata. Ora la parte ordinata è cresciuta di un elemento. Ripete il processo finché l'array non è completamente ordinato.
- **Complessità Temporale:** O(n^2) sempre: nel caso pessimo, medio e ottimo. Deve sempre scansionare tutta la parte rimanente dell'array per trovare il minimo, anche se l'array è già ordinato.

ALGORITMI DI ORDINAMENTO: SELECTION SORT

- **Caratteristiche:** Il suo grande vantaggio è che fa il **minor numero possibile di scambi** (al massimo O(n)). È utile se il costo di scrittura in memoria è estremamente elevato rispetto al costo di lettura (situazione rara oggi).
- Generalmente **non è stabile**.
- **Uso pratico:** Situazioni in cui la memoria in scrittura è un collo di bottiglia estremo

ALGORITMI DI ORDINAMENTO: SELECTION SORT IN PYTHON 1/2

```python
# ============================================================
# SELECTION SORT — Ordinamento per selezione
# Idea: divide l'array in parte ordinata (sinistra) e non
# ordinata (destra). Ad ogni passo, trova il minimo nella
# parte non ordinata e lo mette in fondo alla parte ordinata.
# ============================================================

def selection_sort(arr: list) -> list:
    """
    Ordina una lista in-place con l'algoritmo Selection Sort.

    Complessità temporale:
      - Caso migliore:  O(n²)   — sempre n²/2 confronti
      - Caso medio:     O(n²)
      - Caso peggiore:  O(n²)

    Complessità spaziale: O(1)  — ordinamento in-place
    Stabile: No (gli scambi possono alterare l'ordine relativo)
    Nota: fa sempre esattamente n-1 scambi — ottimo quando
          scrivere in memoria è costoso (es. memoria flash).
    """
    n = len(arr)

    for i in range(n):
        # Supponiamo che il minimo sia il primo elemento
        # della parte non ancora ordinata
        idx_min = i
```

ALGORITMI DI ORDINAMENTO: SELECTION SORT IN PYTHON 2/2

```
            # Cerca il minimo vero nella parte non ordinata
            for j in range(i + 1, n):
                if arr[j] < arr[idx_min]:
                    idx_min = j   # aggiorna l'indice del minimo

            # Scambia il minimo trovato con il primo elemento
            # della parte non ordinata (solo se necessario)
            if idx_min != i:
                arr[i], arr[idx_min] = arr[idx_min], arr[i]

        return arr


# ─── Esempio d'uso ──────────────────────────────────────────
if __name__ == "__main__":
    dati = [64, 25, 12, 22, 11]
    print(f"Prima:  {dati}")
    selection_sort(dati)
    print(f"Dopo:   {dati}")
    # Output:
    # Prima:  [64, 25, 12, 22, 11]
    # Dopo:   [11, 12, 22, 25, 64]
```

ALGORITMI DI ORDINAMENTO: INSERTION SORT

Ordinamento per Inserzione ed è il modo in cui la maggior parte delle persone ordina le carte da gioco in mano. Anche qui dividiamo l'array in parte ordinata e parte da ordinare.

- **Come funziona:** Prende il primo elemento della parte non ordinata e lo "inserisce" nella posizione corretta all'interno della parte già ordinata, facendogli spazio spostando gli elementi più grandi verso destra.
- **Complessità Temporale:** O(n^2) nel caso pessimo e medio (es. array ordinato al contrario). Tuttavia, il caso ottimo (array già ordinato) è **O(n)**. Inoltre, è estremamente efficiente su array quasi ordinati o array molto piccoli.

ALGORITMI DI ORDINAMENTO: INSERTION SORT

- **Caratteristiche:** Fa molti spostamenti (shift) ma meno confronti medi rispetto al Bubble Sort. È un algoritmo **stabile**.
- **Uso pratico:** Altissimo. Paradossalmente, proprio per la sua efficienza su array minuscoli (solitamente meno di 10-20 elementi), algoritmi avanzati come il Quick Sort o il Merge Sort passano all'Insertion Sort quando i sotto-problemi diventano sufficientemente piccoli. (Questo algoritmo ibrido si chiama *TimSort* ed è l'algoritmo di ordinamento di default in Python e Java per gli oggetti).

ALGORITMI DI ORDINAMENTO: INSERTION SORT IN PYTHON 1/2

```python
# ============================================================
# INSERTION SORT — Ordinamento per inserimento
# Idea: costruisce la parte ordinata un elemento alla volta,
# inserendo ogni nuovo elemento nella posizione giusta,
# come si ordinano le carte da gioco in mano.
# ============================================================

def insertion_sort(arr: list) -> list:
    """
    Ordina una lista in-place con l'algoritmo Insertion Sort.

    Complessità temporale:
      - Caso migliore:  O(n)    — lista già ordinata
      - Caso medio:     O(n²)
      - Caso peggiore:  O(n²)   — lista in ordine inverso

    Complessità spaziale: O(1)  — ordinamento in-place
    Stabile: Sì
    Nota: molto efficiente su input piccoli o quasi ordinati.
          È la base del Timsort per le "run" corte.
    """
    # Il primo elemento è già "ordinato" da solo.
    # Partiamo dall'indice 1 e inseriamo ogni elemento
    # nella sottosequenza già ordinata alla sua sinistra.
    for i in range(1, len(arr)):
        # Salviamo il valore da inserire
        chiave = arr[i]
```

ALGORITMI DI ORDINAMENTO: INSERTION SORT IN PYTHON 2/2

```
        # j punta all'ultimo elemento della parte ordinata
        j = i - 1

        # Scorri la parte ordinata verso sinistra, spostando
        # gli elementi maggiori di 'chiave' di una posizione.
        while j >= 0 and arr[j] > chiave:
            arr[j + 1] = arr[j]   # sposta l'elemento a destra
            j -= 1

        # Inserisci 'chiave' nella posizione corretta
        arr[j + 1] = chiave

    return arr


# ─── Esempio d'uso ──────────────────────────────────────────
if __name__ == "__main__":
    dati = [12, 11, 13, 5, 6]
    print(f"Prima:  {dati}")
    insertion_sort(dati)
    print(f"Dopo:   {dati}")
    # Output:
    # Prima:  [12, 11, 13, 5, 6]
    # Dopo:   [5, 6, 11, 12, 13]
```

ALGORITMI DI ORDINAMENTO: TIMSORT 1/5

```
# ============================================================
# TIMSORT — Algoritmo ibrido (Tim Peters, 2002)
# Usato da Python (list.sort), Java (Arrays.sort), Android...
#
# Idea: combina Insertion Sort e Merge Sort sfruttando le
# sequenze già ordinate ("run") presenti nei dati reali.
#
# Passi principali:
#   1. Dividi l'array in "run" di dimensione MIN_RUN (32-64).
#   2. Ordina ogni run con Insertion Sort (ottimale su liste corte).
#   3. Fondi le run con Merge Sort in modo bilanciato.
# ============================================================

MIN_RUN = 32   # dimensione minima di una run (costante di Timsort)


def insertion_sort_run(arr: list, sinistra: int, destra: int) -> None:
    """
    Insertion Sort su una sottosequenza arr[sinistra..destra] (inclusi).
    Usato internamente da Timsort per ordinare le singole run.
    """
    for i in range(sinistra + 1, destra + 1):
        chiave = arr[i]
        j = i - 1
        while j >= sinistra and arr[j] > chiave:
            arr[j + 1] = arr[j]
```

ALGORITMI DI ORDINAMENTO: TIMSORT 2/5

```
            j -= 1
        arr[j + 1] = chiave


def merge(arr: list, sinistra: int, centro: int, destra: int) -> None:
    """
    Fonde due sottosequenze ordinate:
      arr[sinistra..centro]  e  arr[centro+1..destra]
    in una singola sequenza ordinata.
    Usa O(n) spazio extra per le due metà temporanee.
    """
    # Copia le due metà in array temporanei
    len1 = centro - sinistra + 1
    len2 = destra - centro
    metà_sx = arr[sinistra : centro + 1]
    metà_dx = arr[centro + 1 : destra + 1]

    # Tre indici: i per metà_sx, j per metà_dx, k per arr
    i = j = 0
    k = sinistra

    # Confronta e copia il minore dei due fronti
    while i < len1 and j < len2:
        if metà_sx[i] <= metà_dx[j]:
            arr[k] = metà_sx[i]
            i += 1
        else:
```

ALGORITMI DI ORDINAMENTO: TIMSORT 3/5

```
            arr[k] = metà_dx[j]
            j += 1
        k += 1

    # Copia gli elementi rimasti (al più uno dei due è non vuoto)
    while i < len1:
        arr[k] = metà_sx[i]; i += 1; k += 1
    while j < len2:
        arr[k] = metà_dx[j]; j += 1; k += 1


def timsort(arr: list) -> list:
    """
    Ordina una lista con Timsort.

    Complessità temporale:
      - Caso migliore:  O(n)        — lista già ordinata
      - Caso medio:     O(n log n)
      - Caso peggiore:  O(n log n)

    Complessità spaziale: O(n)  — per gli array temporanei del merge
    Stabile: Sì
    Nota: Python usa built-in list.sort() / sorted() che sono Timsort.
          Questa implementazione è didattica e mostra i concetti chiave.
    """
    n = len(arr)
```

ALGORITMI DI ORDINAMENTO: TIMSORT 4/5

```
# ── Fase 1: ordina ogni run con Insertion Sort ──────────────────
# Scorre l'array a passi di MIN_RUN e ordina ogni blocco.
for inizio in range(0, n, MIN_RUN):
    fine = min(inizio + MIN_RUN - 1, n - 1)
    insertion_sort_run(arr, inizio, fine)

# ── Fase 2: fondi le run con Merge raddoppiando la dimensione ─
# La dimensione delle run da fondere raddoppia ad ogni livello:
# MIN_RUN → 2*MIN_RUN → 4*MIN_RUN → ... fino a coprire tutto.
dim = MIN_RUN
while dim < n:
    # Fondi coppie di run adiacenti di dimensione 'dim'
    for sinistra in range(0, n, 2 * dim):
        # Centro: fine della prima run
        centro = min(sinistra + dim - 1, n - 1)
        # Destra: fine della seconda run (può essere più corta)
        destra = min(sinistra + 2 * dim - 1, n - 1)

        # Fondi solo se esiste davvero una seconda run
        if centro < destra:
            merge(arr, sinistra, centro, destra)

    dim *= 2    # raddoppia la dimensione delle run da fondere

return arr
```

ALGORITMI DI ORDINAMENTO: TIMSORT 5/5

```
# ─── Esempio d'uso ──────────────────────────────────────────
if __name__ == "__main__":
    import random
    dati = random.sample(range(1, 101), 20)
    print(f"Prima:  {dati}")
    timsort(dati)
    print(f"Dopo:   {dati}")

    # Confronto con il Timsort nativo di Python
    dati2 = dati.copy()
    dati2.sort()
    print(f"Uguale a sorted(): {dati == dati2}")
```

ALGORITMI DI ORDINAMENTO: QUICK SORT

È l'algoritmo di ordinamento più utilizzato in pratica (spesso integrato nelle librerie standard di molti linguaggi) perché, nonostante il suo caso pessimo sia teoricamente O(n^2), nel caso medio e pratico le costanti nascoste sono bassissime, rendendolo estremamente veloce. Opera in loco (non richiede array extra, a parte lo stack delle chiamate ricorsive), ma non è stabile.

- Scelta del pivot: È il cuore del Quick Sort. Il pivot è un elemento scelto dall'array che farà da "spartiacque".
  - Se scegli sempre il primo o l'ultimo elemento e l'array è già ordinato, finisci nel caso pessimo O(n^2).

ALGORITMI DI ORDINAMENTO: QUICK SORT

- Le strategie moderne prevedono la scelta di un pivot casuale (Randomized Quick Sort) o la tecnica "Mediana di tre" (si guarda il primo, il centrale e l'ultimo elemento, e si sceglie il valore mediano). Una buona scelta del pivot garantisce che l'array venga diviso in due metà quasi uguali, mantenendo la complessità a O(n log n).

- Partizionamento: Una volta scelto il pivot, l'algoritmo riorganizza l'array in modo che tutti gli elementi minori del pivot finiscano alla sua sinistra, e tutti i maggiori alla sua destra. Alla fine di questa fase, il pivot si trova nella sua posizione finale definitiva. A questo punto, l'algoritmo richiama se stesso ricorsivamente sulla sotto-metà sinistra e sulla sotto-metà destra.

ALGORITMI DI ORDINAMENTO: QUICK SORT IN PYTHON 1/4

```
# ============================================================
# QUICKSORT — Ordinamento rapido (C. A. R. Hoare, 1959)
# Idea: "divide et impera" in-place.
#   1. Scegli un elemento "pivot".
#   2. Partiziona l'array: tutti i minori a sinistra del pivot,
#      tutti i maggiori a destra (fase di partition).
#   3. Ricorri sulle due sottosequenze.
#
# Il pivot finisce nella sua posizione definitiva dopo ogni
# partizione → non serve un array extra.
# ============================================================

def _partition(arr: list, basso: int, alto: int) -> int:
    """
    Schema di partizione di Lomuto.
    Sceglie arr[alto] come pivot, poi riorganizza
    arr[basso..alto] in modo che:
      - tutti gli elementi < pivot  stiano a sinistra
      - il pivot stia nella posizione finale
      - tutti gli elementi >= pivot stiano a destra
    Restituisce l'indice finale del pivot.
    """
    pivot = arr[alto]    # pivot = ultimo elemento
    i = basso - 1         # i = indice dell'ultimo "piccolo" trovato
```

ALGORITMI DI ORDINAMENTO: QUICK SORT IN PYTHON 2/4

```
        for j in range(basso, alto):
            if arr[j] <= pivot:
                i += 1
                # Porta arr[j] nella zona dei "piccoli"
                arr[i], arr[j] = arr[j], arr[i]

        # Metti il pivot nella sua posizione corretta
        arr[i + 1], arr[alto] = arr[alto], arr[i + 1]
        return i + 1   # indice definitivo del pivot


def quicksort(arr: list, basso: int = 0, alto: int = None) -> list:
    """
    Ordina arr[basso..alto] in-place con Quicksort ricorsivo.

    Complessità temporale:
      - Caso migliore:  O(n log n) — pivot sempre mediano
      - Caso medio:     O(n log n) — con pivot casuale o random
      - Caso peggiore:  O(n²)      — array già ordinato + pivot fisso

    Complessità spaziale: O(log n) medio (stack ricorsivo)
                          O(n) peggiore
    Stabile: No
    In-place: Sì (nessun array aggiuntivo)
    """
    if alto is None:
        alto = len(arr) - 1
```

ALGORITMI DI ORDINAMENTO: QUICK SORT IN PYTHON 3/4

```
    # Caso base: sottoarray di 0 o 1 elementi → già ordinato
    if basso < alto:
        # Partiziona e ottieni l'indice finale del pivot
        idx_pivot = _partition(arr, basso, alto)

        # Ricorri sulla metà sinistra (escluso il pivot)
        quicksort(arr, basso, idx_pivot - 1)
        # Ricorri sulla metà destra (escluso il pivot)
        quicksort(arr, idx_pivot + 1, alto)

    return arr


# ─── Variante con pivot casuale (consigliata in pratica) ──────
# Scegliere il pivot a caso evita il caso peggiore O(n²)
# su input già ordinati o con molti duplicati.

import random

def _partition_random(arr: list, basso: int, alto: int) -> int:
    """Partiziona con pivot scelto casualmente nell'intervallo."""
    idx_casuale = random.randint(basso, alto)
    # Porta il pivot in coda per usare lo schema Lomuto
    arr[idx_casuale], arr[alto] = arr[alto], arr[idx_casuale]
    return _partition(arr, basso, alto)

]
```

ALGORITMI DI ORDINAMENTO: QUICK SORT IN PYTHON 4/4

```python
def quicksort_random(arr: list, basso: int = 0, alto: int = None) -> list:
    """Quicksort con pivot casuale: O(n log n) atteso sempre."""
    if alto is None:
        alto = len(arr) - 1
    if basso < alto:
        idx_pivot = _partition_random(arr, basso, alto)
        quicksort_random(arr, basso, idx_pivot - 1)
        quicksort_random(arr, idx_pivot + 1, alto)
    return arr


# ─── Esempio d'uso ──────────────────────────────────────────
if __name__ == "__main__":
    dati = [10, 7, 8, 9, 1, 5]
    print(f"Prima:  {dati}")
    quicksort(dati)
    print(f"Dopo:   {dati}")
    # Output:
    # Prima:  [10, 7, 8, 9, 1, 5]
    # Dopo:   [1, 5, 7, 8, 9, 10]
```

ALGORITMI DI ORDINAMENTO: MERGE SORT

È l'algoritmo "Divide et Impera" per eccellenza. Il suo pregio più grande è l'assoluta prevedibilità: costa **sempre O(n log n)**, sia nel caso pessimo, medio che ottimo. Inoltre, è un algoritmo **stabile**.
Il suo svantaggio principale è che **non opera in loco**: richiede una memoria ausiliaria pari a O(n) per unire i sotto-array.

- **Fase di divisione:** È la parte banale. L'array viene diviso a metà, poi le metà vengono divise a metà, e così via in modo ricorsivo finché non si ottengono n sotto-array di dimensione 1. (Un array di 1 elemento è, per definizione, ordinato).

ALGORITMI DI ORDINAMENTO: MERGE SORT

- **Fase di merge (fusione):** È qui che avviene il vero lavoro. L'algoritmo prende due sotto-array già ordinati e li fonde in un unico array ordinato più grande. Per farlo, usa due "puntatori" che scorrono i due sotto-array: confronta gli elementi puntati, prende il più piccolo, lo copia nell'array ausiliario e fa avanzare il puntatore corrispondente. Questa fusione a cascata continua finché l'intero array non è ricomposto.

ALGORITMI DI ORDINAMENTO: MERGE SORT IN PYTHON 1/4

```
# ============================================================
# MERGE SORT — Ordinamento per fusione (John von Neumann, 1945)
# Idea: "divide et impera" con array ausiliari.
#   1. Dividi l'array a metà ricorsivamente fino ad avere
#      sottoarray di un solo elemento (già ordinati per def.).
#   2. Fondi (merge) coppie di sottoarray ordinati in uno
#      solo ordinato, risalendo la ricorsione.
#
# Garanzia: O(n log n) in tutti i casi → prediletto quando
# la stabilità o il worst-case certo sono requisiti.
# ============================================================

def _merge(sinistra: list, destra: list) -> list:
    """
    Fonde due liste ordinate in una nuova lista ordinata.
    Confronta i fronti delle due liste e aggiunge il minore.
    Complessità: O(n+m) dove n, m sono le lunghezze.
    """
    risultato = []
    i = j = 0

    # Confronta gli elementi testa a testa
    while i < len(sinistra) and j < len(destra):
        if sinistra[i] <= destra[j]:   # <= garantisce stabilità
            risultato.append(sinistra[i])
            i += 1
```

ALGORITMI DI ORDINAMENTO: MERGE SORT IN PYTHON 2/4

```
            else:
                risultato.append(destra[j])
                j += 1

        # Appende i rimanenti (al più una lista ha ancora elementi)
        risultato.extend(sinistra[i:])
        risultato.extend(destra[j:])
        return risultato


def mergesort(arr: list) -> list:
    """
    Ordina una lista con Merge Sort (versione top-down ricorsiva).
    Restituisce una NUOVA lista ordinata (non modifica l'originale).

    Complessità temporale:
      - Caso migliore:  O(n log n)
      - Caso medio:     O(n log n)
      - Caso peggiore:  O(n log n)  ← garantito sempre

    Complessità spaziale: O(n) — per i sottoarray ausiliari
                          O(log n) stack ricorsivo
    Stabile: Sì (grazie al <= nel confronto)
    In-place: No
    """
```

ALGORITMI DI ORDINAMENTO: MERGE SORT IN PYTHON 3/4

```
    # Caso base: 0 o 1 elementi → già ordinato
    if len(arr) <= 1:
        return arr

    # ── Fase DIVIDE: trova il punto di mezzo e dividi ──────────
    meta = len(arr) // 2
    metà_sx = mergesort(arr[:meta])    # ordina metà sinistra
    metà_dx = mergesort(arr[meta:])    # ordina metà destra

    # ── Fase IMPERA: fondi le due metà ordinate ────────────────
    return _merge(metà_sx, metà_dx)


# ─── Variante bottom-up (iterativa, senza ricorsione) ─────────
# Evita l'overhead dello stack ricorsivo:
# parte da sottoarray di dim=1 e raddoppia ad ogni livello.

def mergesort_iterativo(arr: list) -> list:
    """
    Merge Sort bottom-up: stessa complessità, zero ricorsione.
    Lavora direttamente su arr (modifica in-place logicamente
    usando slice assignment).
    """
    n = len(arr)
    dim = 1   # dimensione corrente dei blocchi da fondere
```

ALGORITMI DI ORDINAMENTO: MERGE SORT IN PYTHON 4/4

```
    while dim < n:
        for sinistra in range(0, n, 2 * dim):
            centro = min(sinistra + dim, n)
            destra = min(sinistra + 2 * dim, n)
            # Fondi arr[sinistra:centro] con arr[centro:destra]
            arr[sinistra:destra] = _merge(
                arr[sinistra:centro],
                arr[centro:destra]
            )
        dim *= 2

    return arr


# ─── Esempio d'uso ──────────────────────────────────────────
if __name__ == "__main__":
    dati = [38, 27, 43, 3, 9, 82, 10]
    print(f"Prima:       {dati}")
    ordinata = mergesort(dati)
    print(f"Dopo (nuova): {ordinata}")
    print(f"Originale:    {dati}")   # invariato!
    # Output:
    # Prima:        [38, 27, 43, 3, 9, 82, 10]
    # Dopo (nuova): [3, 9, 10, 27, 38, 43, 82]
    # Originale:    [38, 27, 43, 3, 9, 82, 10]
```

ALGORITMI DI ORDINAMENTO: HEAP SORT

L'Heap Sort abbandona il "Divide et Impera" puro e si affida all'efficienza di una struttura dati ad albero chiamata **Heap** (precisamente un Max-Heap per ordinare in senso crescente). Un Max-Heap è un albero binario quasi completo in cui il valore di ogni nodo padre è maggiore o uguale a quello dei suoi figli. L'Heap Sort garantisce un tempo O(n log n) nel caso pessimo e opera **in loco**, ma **non è stabile**.

- **Costruzione Heap (Build-Max-Heap):** L'algoritmo prende l'array disordinato e lo riorganizza logicamente per rispettare le proprietà del Max-Heap. Sorprendentemente, questa operazione iniziale si può fare in tempo O(n) partendo dal basso verso l'alto (eseguendo un'operazione chiamata *heapify* sui nodi interni).

ALGORITMI DI ORDINAMENTO: HEAP SORT

- **Fase di ordinamento:** Una volta costruito il Max-Heap, sappiamo per certo che l'elemento più grande in assoluto si trova nella radice (la prima posizione dell'array). L'algoritmo prende questo elemento massimo e lo scambia con l'ultimo elemento dell'array. Ora il numero più grande è al suo posto definitivo alla fine. L'albero rimanente (più piccolo di 1) viene "aggiustato" (*heapify down*) per far risalire il nuovo massimo alla radice. Si ripete finché l'albero non si svuota.

ALGORITMI DI ORDINAMENTO: HEAP SORT IN PYTHON 1/4

```
   # ============================================================
# HEAP SORT — Ordinamento con heap (J. W. J. Williams, 1964)
# Idea: sfrutta la struttura dati "max-heap" (albero binario
# completo dove ogni nodo ≥ figli) rappresentata come array.
#
# Relazione indici (array 0-based):
#   Padre di i:      (i - 1) // 2
#   Figlio sinistro: 2*i + 1
#   Figlio destro:   2*i + 2
#
# Due fasi:
#   1. Heapify   — costruisci il max-heap in O(n)
#   2. Estrazione — estrai il massimo (radice) n volte in O(log n)
# ============================================================

def _sift_down(arr: list, n: int, i: int) -> None:
    """
    Ripristina la proprietà max-heap nell'albero radicato in i,
    considerando solo i primi n elementi di arr.

    "Sift down" = fai scendere arr[i] finché non è ≥ di entrambi
    i figli (o finché non è una foglia).
    """
    massimo = i          # supponiamo la radice sia il massimo
    sx = 2 * i + 1     # figlio sinistro
    dx = 2 * i + 2     # figlio destro
```

ALGORITMI DI ORDINAMENTO: HEAP SORT IN PYTHON 2/4

```
    # Il figlio sinistro esiste ed è maggiore del massimo corrente?
    if sx < n and arr[sx] > arr[massimo]:
        massimo = sx

    # Il figlio destro esiste ed è maggiore del massimo corrente?
    if dx < n and arr[dx] > arr[massimo]:
        massimo = dx

    # Se il massimo non è la radice, scambia e continua a scendere
    if massimo != i:
        arr[i], arr[massimo] = arr[massimo], arr[i]
        _sift_down(arr, n, massimo)   # ricorri sul sottoalbero mosso


def heapsort(arr: list) -> list:
    """
    Ordina una lista in-place con Heap Sort.

    Complessità temporale:
      - Caso migliore:  O(n log n)
      - Caso medio:     O(n log n)
      - Caso peggiore:  O(n log n)  ← garantito sempre

    Complessità spaziale: O(log n) stack ricorsivo (sift_down)
                          O(1) se sift_down iterativo
```

ALGORITMI DI ORDINAMENTO: HEAP SORT IN PYTHON 3/4

```
Stabile: No (i salti lunghi rompono l'ordine relativo)
In-place: Sì
"""
n = len(arr)

# ── Fase 1: BUILD MAX-HEAP ──────────────────────────────────
# Partiamo dall'ultimo nodo interno (foglie sono già heap da sole)
# e applichiamo sift_down verso la radice.
# L'ultimo nodo interno ha indice n//2 - 1.
# Costo totale: O(n) — non O(n log n) come si potrebbe pensare.
for i in range(n // 2 - 1, -1, -1):
    _sift_down(arr, n, i)

# ── Fase 2: ESTRAZIONE ORDINATA ─────────────────────────────
# Il massimo è sempre in arr[0] (radice del max-heap).
# Scambia radice con l'ultimo elemento, "riduci" l'heap di 1,
# poi ripristina la proprietà heap con sift_down.
# Ripetendo n-1 volte, l'array si riempie da destra con
# i massimi in ordine crescente.
for i in range(n - 1, 0, -1):
    # Porta il massimo corrente nella sua posizione finale
    arr[0], arr[i] = arr[i], arr[0]
    # Ripristina il max-heap sui restanti i elementi
    _sift_down(arr, i, 0)

return arr
```

ALGORITMI DI ORDINAMENTO: HEAP SORT IN PYTHON 4/4

```python
# ─── Nota sul modulo heapq di Python ──────────────────────────
# Python offre heapq (min-heap) nella libreria standard.
# Per ottenere un max-heap basta negare i valori.

import heapq

def heapsort_builtin(arr: list) -> list:
    """Versione con heapq della stdlib — idiomatica Python."""
    # heapq.nsmallest è O(n log n) e usa un heap internamente
    return list(heapq.nsmallest(len(arr), arr))


# ─── Esempio d'uso ────────────────────────────────────────────
if __name__ == "__main__":
    dati = [12, 11, 13, 5, 6, 7]
    print(f"Prima:  {dati}")
    heapsort(dati)
    print(f"Dopo:   {dati}")

    # Visualizza il max-heap intermedio (dopo build-heap)
    dati2 = [3, 1, 4, 1, 5, 9, 2, 6]
    n = len(dati2)
    for i in range(n // 2 - 1, -1, -1):
        _sift_down(dati2, n, i)
    print(f"Max-heap: {dati2}")   # [9, 6, 4, 1, 5, 3, 2, 1]
    heapsort(dati2)
    print(f"Ordinato: {dati2}")   # [1, 1, 2, 3, 4, 5, 6, 9]
```

ALGORITMI DI ORDINAMENTO: CONFRONTO

Quick Sort vs Merge Sort

È il confronto più classico. Entrambi usano il *Divide et Impera*, ma con filosofie opposte.

- **Tempo:** Entrambi hanno caso medio O(n log n). Il Merge Sort lo garantisce anche nel caso pessimo, mentre il Quick Sort può degenerare a O(n^2) (se il pivot è scelto male).
- **Spazio:** Quick Sort è **in loco** (usa solo O(log n) per lo stack delle chiamate ricorsive). Merge Sort richiede un array ausiliario di dimensione O(n) per la fusione.
- **Stabilità:** Merge Sort è stabile, Quick Sort no.
- **Chi vince in pratica?** Il **Quick Sort**. Grazie al modo in cui analizza array contigui, ha una **località di riferimento (cache locality)** eccellente a livello di CPU, rendendo le sue costanti nascoste molto più piccole rispetto al Merge Sort.

ALGORITMI DI ORDINAMENTO: CONFRONTO

Quick Sort vs Bubble Sort

Non c'è vera partita, ma serve a capire l'evoluzione.

- Bubble Sort è O(n^2), Quick Sort è mediamente O(n log n). Su array di grandi dimensioni, il Quick Sort finisce in frazioni di secondo ciò che il Bubble Sort impiega ore a fare.
- L'unica magra consolazione del Bubble Sort è che è stabile e, se ottimizzato, impiega O(n) su un array già ordinato (mentre un Quick Sort ingenuo su un array ordinato cade nel suo caso pessimo O(n^2)).

ALGORITMI DI ORDINAMENTO: CONFRONTO

Heap Sort vs Merge Sort vs Quick Sort (I "Big Three")

- **Quick Sort:** Il re della pratica (veloce, in loco), ma instabile e rischioso nel caso pessimo.
- **Merge Sort:** L'affidabile (sempre O(n log n), stabile), ma "sprecone" in termini di memoria (O(n)).
- **Heap Sort:** Il compromesso teorico perfetto (sempre O(n log n) e in loco), ma **instabile e lento in pratica** rispetto al Quick Sort a causa dei continui salti nell'array (scarsa cache locality) per calcolare padri e figli.

ALGORITMI DI ORDINAMENTO: CONFRONTO

Insertion Sort vs Selection Sort

Entrambi O(n^2), ma si comportano diversamente.

- **Insertion Sort** è **adattivo**: se l'array è parzialmente ordinato, interrompe i confronti in anticipo. Selection Sort invece scansiona sempre tutto, costando O(n^2) anche se l'array è già perfetto.
- **Selection Sort** fa al massimo O(n) **scambi**. Insertion Sort fa moltissimi spostamenti in memoria. Se scrivere in memoria costa infinitamente più che leggere (es. memorie flash antiche), vince Selection Sort. In tutti gli altri casi, vince Insertion Sort.

ALGORITMI DI ORDINAMENTO: CONFRONTO

Insertion Sort vs Bubble Sort

Entrambi O(n^2), stabili e adattivi (O(n) nel caso ottimo).

- **Insertion Sort vince a mani basse.** Il Bubble Sort esegue uno scambio completo (3 operazioni di assegnamento) ad ogni passo. L'Insertion Sort fa solo un singolo "shift" (1 operazione di assegnamento) per spostare gli elementi verso destra.

ALGORITMI DI ORDINAMENTO: CONFRONTO

Insertion Sort vs Heap Sort

Perché confrontare un algoritmo O(n^2) con uno O(n log n)? Per una questione di **dimensione dell'input (overhead)**.

- Su n molto grande, Heap Sort vince clamorosamente.
- Su n molto piccolo (generalmente n < 16 o n < 20), **Insertion Sort è più veloce**. Non ha bisogno di creare l'albero logico, non ha chiamate ricorsive, è banalmente un ciclo while compatto che sta tutto nella cache del processore.

ALGORITMI DI ORDINAMENTO: OTTIMIZZAZIONI E VARIANTI

Un piccolo focus sull'Insertion Sort

Molti preferiscono l'Insertion Sort perché è il "migliore tra i peggiori" ed è il cuore di molte ottimizzazioni moderne.

Insertion Sort su Array Quasi Ordinati
Un array "quasi ordinato" è un array in cui ogni elemento si trova a non più di una distanza costante k dalla sua posizione finale corretta, oppure dove il numero di inversioni (coppie disordinate) è molto basso.

- In questo scenario, il ciclo interno dell'Insertion Sort si ferma quasi subito.
- La complessità crolla da O(n^2) a **O(n + d)**, dove d è il numero di inversioni. Se d è proporzionale a n, il tempo di esecuzione diventa **Lineare O(n)**.

ANALISI DEGLI ALGORITMI

Referenze

- Algoritmi e Strutture Dati – Demetrescu, Finocchi, Italiano
- Introduzione agli algoritmi e strutture dati – Cormen, Leiserson, Rivest, Stein

Risorse Python

https://runestone.academy/ns/books/published/pythonds/index.html?utm_source=chatgpt.com

https://github.com/0bprashanthc/algorithm-books/blob/master/Data%20Structures%20%26%20Algorithms%20in%20Python.pdf?utm_source=chatgpt.com

https://edu.anarcho-copy.org/Programming%20Languages/Python/Python%20Data%20Structures%20and%20Algorithms.pdf?utm_source=chatgpt.com

https://do1.dr-chuck.com/pythonlearn/IT_it/pythonlearn.pdf?utm_source=chatgpt.com

